

Milestone Three Narrative

Enhancement Two: Algorithms and Data Structures

Garrett Sikes

9/21/2026

SNHU

CS 499 Computer Science Capstone

Artifact: ProjectTwo.cpp, course advising tool (DSA-AnalysisAndDesign)

ProjectTwo.cpp was a program I wrote in DSA: Analysis and Design (CS 300). The program accepted a CSV file and stored the courses in a binary search tree data structure. Its capabilities were loading information from a CSV file, printing the full course list, and looking up course information (number, name, and prerequisites).

I chose to enhance this artifact for the Algorithms and Data Structures category because of a specific structural weakness in how it stored data. A binary search tree normally splits data to the left and right as it grows, which keeps searches fast. But if the course data comes in already sorted by course number, every new course gets added to the same side instead of splitting the tree, so it ends up as a straight line instead of a balanced shape. When that happens, the program has to check each course one at a time instead of skipping half the remaining courses with every comparison, which is much slower. In testing, the unbalanced version was about 100 times slower to load the data and about 400 times slower to look up a course compared to the AVL version. Converting the tree to a self-balancing AVL tree fixes this by rebalancing itself with every course added, so it stays fast no matter what order the data comes in.

While making the AVL change, I also found two smaller issues in the code. The CSV parser wasn't trimming extra whitespace or checking for a blank course name, so a malformed row could silently load as if it were a valid course. I fixed it so rows missing a course number or name are rejected and flagged instead. I also found that the original file was missing a required `#include <limits>` statement, which kept it from compiling at all on some systems even though it worked in the environment it was originally written in. I added that include to fix the portability problem.

In my Module One plan, I framed this enhancement as an extension of the same security mindset running through all three of my chosen artifacts: making the course advising program catch bad or malformed input instead of quietly corrupting the tree with it. The `parseCourse()` fix delivers on that directly, since it now rejects incomplete or badly formatted course records before they ever reach the tree, instead of silently accepting them. The AVL conversion was also part of that original plan, and looking at the finished work, it clearly satisfies a second course outcome as well: designing and evaluating a computing solution using algorithmic principles, including managing the trade-offs involved. Implementing all four AVL rotation cases correctly shows that directly, and the numbers back it up: the unbalanced tree was up to 400 times slower on sorted data, while the AVL tree performed consistently well in both cases.

Working on this enhancement taught me that a binary search tree, while not a bad choice in general, isn't the most efficient option when the data comes in already sorted. For example, if you had a course like CS 499, a plain BST would have to walk through nearly every earlier CS course to reach it, since sorted input builds a long chain instead of a balanced tree. An AVL tree only has to check a small fraction of the courses to find the same one, and it rebalances itself automatically as more courses are added, so it stays efficient no matter how the data comes in.

The general concept of self-balancing made sense once I worked through it, but tracing exactly how a single rotation rebalances the tree without breaking the sorted order, and figuring out which of the four rotation cases applies in a given situation, was harder to hold in my head than to read about. What helped was working through the logic case by case instead of trying to memorize the whole algorithm at once.